

PENUMBRA

High-Level Design (HLD)

Document ID: PEN-HLD-001

Version: 1.0

Date: 23 July 2026

Traces to: PEN-SRS-001

1. System Overview

PENUMBRA is a two-tier web system — a Python forecasting/orchestration backend and a React charting frontend — arranged around a **cache-first data pipeline**. Public NOAA/GFZ feeds are polled on schedule, normalised into aligned time series, run through a leakage-free walk-forward backtest to learn per-lead-time error distributions, and then combined with a point model to emit **probabilistic forecasts with verified calibration**. The pipeline is modelled as six named **agents**; the same graph is rendered as the n8n-style diagram embedded in the UI.

2. Architecture



2.1 Component Responsibilities

Component	Layer	Responsibility	SRS trace
swpc_client	service	Fetch F10.7, Kp, 45-day forecast, long history from NOAA (keyless); retry + backoff; typed <code>DataSourceError</code> .	FR-ING-1..3
cache_service	service	SQLite key/TTL cache + forecast read-model + pipeline runs; last-known-good fallback; AWS swap seam.	FR-ING-4/6
forecast_service	service	Pipeline orchestrator: load → align → backtest → forecast → calibrate → publish. Read model for API.	all FR-*
pipeline_service	service	Per-agent stage status/timings powering the live diagram.	FR-UI-4
briefing_service	service	Optional Claude space-weather briefing; deterministic template fallback.	—
series	logic	Load/align/resample/gap-fill the F10.7 and Kp series; daily aggregation of Kp→Ap.	FR-ING-2
f107_forecast	logic	Point forecast (persistence + reversion to 27-day + climatology) and quantile bands from backtest errors.	FR-F107-*
kp_forecast	logic	Recency-weighted categorical storm probabilities; expected Ap band.	FR-KP-*
calibration	logic	Walk-forward backtest, coverage, reliability, pinball loss, skill vs NOAA.	FR-VER-*
drag	logic	F10.7 band → density band (reduced model) → along-track/decay uncertainty.	FR-DRAG-*

2.2 The Agent Pipeline

#	Agent	Trigger	Function
1	SWPC Sync	Startup + 3 h	Pull F10.7, Kp, NOAA 45-day forecast, long history → cache.
2	History Builder	On new data	Align to a daily grid, aggregate Kp→daily max/Ap, gap-fill.
3	Backtest Engine	On history	Walk-forward, leakage-free; per-lead-time error distribution + Kp transition stats.
4	Forecast Engine	Per issue	Point F10.7 + quantile bands; Kp category probabilities; expected Ap.
5	Calibration Auditor	On backtest	Coverage, reliability, pinball, skill vs NOAA; flags mis-calibration.
6	Publisher	On snapshot	Persist read-model (forecast + drag + calibration), expose via REST, notify UI.

3. Forecasting Approach (rationale)

PENUMBRA deliberately uses a **transparent, defensible statistical model** rather than an opaque black box, because the deliverable is *calibrated uncertainty*, which must be verifiable:

- **F10.7 point:** F10.7 is strongly autocorrelated day-to-day and modulated by the 27-day solar-rotation period. The point forecast blends short-lag persistence with mean-reversion toward the trailing 27-day mean and the longer cycle level, the reversion strength increasing with lead time. This matches the physical behaviour and is a strong, honest baseline.
- **Uncertainty:** the band at lead L is the point forecast plus the empirical p05...p95 of the *out-of-sample* forecast errors at lead L, measured by walk-forward backtest. Because the band is learned from real forecast errors, it is calibrated by construction if the backtest is leakage-free — which is the single most important correctness property (CON-2, FR-VER-1).
- **Kp:** a bounded quasi-discrete index; a point value is misleading. PENUMBRA emits category probabilities from recency-weighted historical transition frequencies conditioned on the current activity level — an operationally meaningful "chance of a storm" per day.
- **Skill:** every claim is checked against NOAA's own 45-day forecast, so "better than the official product" is a measured, reproducible statement, not a marketing one.

Honest scope. This is a statistically calibrated reduced-model forecaster, not a physics ensemble. The value is that it publishes *verified* uncertainty where the operational products publish none — exactly the gap identified by Parker et al. (2024).

4. Data Design (summary)

SQLite (`backend/app/data/cache.db`): `kv_cache` (source payloads + TTL), `forecast_snapshots` (issued forecast + calibration read-model as JSON documents), and `pipeline_runs` (agent, timing, status, items, error). The `CacheService` interface is the single seam for a later move to managed storage.

5. Error Handling & Logging

- Exception hierarchy rooted at `PenumbraError(code, message, http_status)` :
`DataSourceError` (502), `InsufficientDataError` (422), `ForecastError` (500),
`NotFoundError` (404), `ConfigError` (500), `ValidationFailure` (422).
- Boundary rule: logic raises; service contextualises & decides fallback (cache, demo, skip); API converts to the JSON error envelope. The UI never sees a stack trace.
- Structured logging (request_id middleware); agent runs logged start/finish with duration and item counts. WARN = degraded (fallback used), ERROR = user-visible failure.

6. Technology Choices

Choice	Rationale
FastAPI + Pydantic v2	Typed contracts, free OpenAPI, async polite fetching.
NumPy only (no ML framework)	The method is quantile statistics + a linear point model; heavyweight ML deps add risk and memory without adding calibration.
SQLite	Zero-ops local persistence; interface-isolated for later cloud storage.
React + Canvas charts (no chart lib)	Fan charts with nested bands are hand-drawn on Canvas for full control and a tiny bundle; consistent with the dataviz principles.
Vite	Fast dev server, <code>/api</code> proxy to backend.

7. Deployment View

v1.0: localhost (uvicorn :8001 + Vite :5175 proxy) and the shared Lightsail VPS (13.127.244.0). On the VPS, PENUMBRA runs as its own systemd service on **port 8001**, exposed through an isolated nginx server block on a dedicated public port, with **KESSLER's port-80 default server left entirely untouched** (NFR-5). A bounded `MemoryMax` and the host's existing swap keep the two services co-resident on 414 MB. **v2.0:** managed storage/scheduler behind the `CacheService` seam; no logic-layer change anticipated.

8. Risks & Mitigations

Risk	Impact	Mitigation
In-sample band fitting (leakage)	Over-confident, mis-calibrated bands	Strict walk-forward split; a leakage-detection unit test.
Shared-host memory pressure	OOM affecting KESSLER	MemoryMax cap; in-process caches; swap; monitored on deploy.
Reduced drag model over-read	Users treat it as operational CA	Explicit "reduced-model, order-of-magnitude" labelling in UI + API.
Sparse Kp transition history for rare storms	Noisy storm probabilities	Recency-weighting + smoothing; wide honest bands; documented limitation.